DAM 0485 · Programación · Unidad Didáctica 4

Boletín 4.2: Nivel Intermedio — Ejercicios Prácticos

Ejercicios avanzados de orientación a objetos: composición, contratos equals y hashCode, inmutabilidad, records, enums, patrón Singleton y factorías estáticas en Java.

Bienvenido al boletín de nivel intermedio de la **Unidad 4: Clases y objetos**.

Este bloque de 16 ejercicios está diseñado para profundizar en la arquitectura y buenas prácticas del paradigma orientado a objetos: diseño de clases compuestas, implementación rigurosa de los contratos de `java.lang.Object` (`toString`, `equals`, `hashCode`), clases inmutables con copias defensivas, los modernos `record`, enumeraciones complejas con comportamiento, el patrón de diseño Singleton, procesamiento con `StringBuilder` y expresiones regulares.

> **Enfoque Arquitectural**
>
> Presta especial atención a la protección de datos: evita las fugas de encapsulamiento mediante copias defensivas (`.clone()` / `Arrays.copyOf()`) y asegura que cualquier clase que sobrescriba `equals()` sobrescriba de forma coordinada `hashCode()`.

Ejercicio 1: Sobrescritura Canónica de `toString()`, `equals()` y `hashCode()` en `Libro`

Enunciado:
Toma como base la clase `Libro` (`titulo`, `autor`, `paginas`):

- Sobrescribe `toString()` con `@Override` para que devuelva el formato `"Ficciones, de Borges (200 páginas)"`. Prueba a pasar directamente un objeto `Libro` a `System.out.println(libro)`.
- Sobrescribe `equals(Object o)` y `hashCode()` de forma que dos objetos `Libro` se consideren iguales si tienen exactamente el mismo `titulo` y el mismo `autor` (aunque tengan diferente número de páginas).
- En `main`, prueba explícitamente estos **tres casos obligatorios**:
  1. Dos objetos distintos con igual título y autor pero distintas páginas → `equals()` debe devolver `true`.
  2. Dos objetos con distinto título → `equals()` debe devolver `false`.
  3. Un objeto comparado consigo mismo (`a.equals(a)`) → debe devolver `true` (propiedad reflexiva).

Ejercicio 2: Composición de Objetos (`Motor` y `Vehiculo`)

Enunciado:
La composición (*"un objeto tiene otro objeto"*) es una de las relaciones más potentes de la POO:

- Crea la clase `Motor` con: `tipoCombustible` (String: Gasolina, Diésel, Eléctrico), `potenciaCv` (int) y `encendido` (boolean). Incluye métodos para encender, apagar y consultar estado.
- Crea la clase `Vehiculo` que contenga: `matricula` (String), `marca` (String) y un atributo de tipo objeto `Motor`.
- El constructor de `Vehiculo` debe instanciar o recibir su `Motor`.
- Proporciona métodos en `Vehiculo` para arrancar, detener y consultar la información completa delegando en su motor interno.

Ejercicio 3: La Clase `Racional` / `Fraccion` Inmutable con Simplificación MCD

Enunciado:
Diseña una clase inmutable llamada `Fraccion` (o `Racional`) definida por dos enteros `numerador` y `denominador` (distinto de cero):

- Incluye un constructor por defecto que genere el racional `1/1`, y un constructor parametrizado que lance `IllegalArgumentException` si `denominador == 0`, normalizando el signo y simplificando la fracción automáticamente mediante el **Máximo Común Divisor (algoritmo de Euclides)**.
- Implementa métodos de instancia `sumar(Fraccion otra)`, `restar(Fraccion otra)`, `multiplicar(Fraccion otra)` y `dividir(Fraccion otra)` que devuelvan siempre una **nueva instancia**, además de un método estático `Fraccion.sumar(Fraccion r1, Fraccion r2)`.
- Sobrescribe `toString()` (`"n/d"`) y `equals()` / `hashCode()`.

Ejercicio 4: Clase Inmutable Clásica `Coordenada` y `record CoordenadaGPS`

Enunciado:
Practica las dos formas de crear tipos inmutables de coordenadas en Java:

1. **Enfoque tradicional:** Crea una clase `public final class Coordenada` con atributos `private final double x` e `y`, sin setters, con getters y un método `public double distanciaA(Coordenada otra)` que calcule la distancia euclídea: `Math.sqrt(Math.pow(otra.x - this.x, 2) + Math.pow(otra.y - this.y, 2))`.
2. **Enfoque Java moderno:** Declara un `record CoordenadaGPS(double latitud, double longitud)` con constructor compacto que valide los rangos `[-90, 90]` y `[-180, 180]` y un método `distanciaKm(CoordenadaGPS destino)` mediante la fórmula de Haversine.

Ejercicio 5: El Patrón Singleton (`ContadorGlobal` y `GestorConfiguracion`)

Enunciado:
Implementa el patrón de diseño Singleton en dos variantes:

1. **Inicialización perezosa (*Lazy*):** Crea la clase `ContadorGlobal` con constructor `private`, instancia estática única y métodos `getInstancia()`, `incrementar()` y `getValor()`. Comprueba en `main` que incrementarlo desde dos referencias distintas (`c1` y `c2`) afecta al mismo contador y que `c1 == c2` es `true`.
2. **Inicialización temprana (*Eager*):** Crea la clase `GestorConfiguracion` que centralice un mapa de propiedades clave-valor (`setPropiedad`, `getPropiedad`).

Ejercicio 6: La Clase `Mochila` y Copias Defensivas (`.clone()` y `Arrays.copyOf()`)

Enunciado:
Diseña una clase `Mochila` que almacene un array `private String[] objetos`:

- Tanto en el constructor al recibir el array como en el getter `getObjetos()`, realiza una **copia defensiva** usando `.clone()` o `Arrays.copyOf()` para que nadie pueda alterar el array interno desde fuera.
- En `main`, crea un array `String[] equipo = {"Linterna", "Cantimplora"}`, pásalo al constructor de `Mochila`, modifica después `equipo[0] = "ROTO"` y altera también el array devuelto por `getObjetos()[1] = "VACÍO"`. Imprime el contenido real de la mochila para demostrar que sigue intacto.

Ejercicio 7: Enumeración Rica `NivelSeveridad`

Enunciado:
Modela una enumeración para un sistema de logging profesional llamada `NivelSeveridad`:

- Constantes: `DEBUG(1, "Depuración"), INFO(2, "Informativo"), WARN(3, "Advertencia"), ERROR(4, "Error Grave"), FATAL(5, "Fallo Crítico")`.
- Atributos privados: `int nivelNumerico` y `String etiqueta`.
- Constructor de enum parametrizado.
- Método `public boolean esCritico()`: devuelve `true` si el nivel es `ERROR` o `FATAL`.
- Método `public boolean esMayorOIgualQue(NivelSeveridad otro)`: compara si la severidad actual es igual o más prioritaria que otra.

Ejercicio 8: `CuentaBancaria` con Validación de Excepciones e Historial

Enunciado:
Crea una clase `CuentaBancaria` con `iban`, `titular`, `saldo` y un registro de las últimas 10 transacciones (`record Transaccion`):

- `ingresar(double cantidad)`: lanza `IllegalArgumentException` si `cantidad <= 0`.
- `retirar(double cantidad)`: lanza `IllegalArgumentException` con un mensaje descriptivo si `cantidad <= 0` o si `cantidad > saldo` (`"Saldo insuficiente"`).
- En `main`, envuelve las operaciones en un bloque `try-catch (IllegalArgumentException e)` intentando retirar más dinero del disponible y muestra el historial de movimientos.

Ejercicio 9: Validación Compacta con `record Usuario`

Enunciado:
Crea un `record Usuario(String username, String email, int edad)`:

- En el constructor compacto, valida:
  1. `username`: alfanumérico entre 4 y 15 caracteres (mediante regex `^[a-zA-Z0-9_]{4,15}$`).
  2. `email`: formato de correo válido con regex.
  3. `edad`: mayor o igual a 18 años.
- Si algún dato no cumple, lanza `IllegalArgumentException` con un mensaje explicativo.

Ejercicio 10: Enumeración de Divisas con Conversión Monetaria

Enunciado:
Crea un `enum Divisa`:

- Constantes: `EUR(1.0, "€"), USD(1.08, "$"), GBP(0.85, "£"), JPY(162.50, "¥")`.
- Cada divisa almacena su tasa de cambio respecto a 1.0 Euro base y su símbolo monetario.
- Diseña un método en el enum `public double convertirA(double cantidad, Divisa divisaDestino)` que transforme cualquier importe entre dos divisas cualesquiera pasando por el Euro como pivote.

Ejercicio 11: Encapsulación de Matriz con la Clase `Matriz2D`

Enunciado:
Crea una clase `Matriz2D` que encapsule una matriz bidimensional `double[][]`:

- Constructor con número de filas y columnas, rellenando con ceros.
- Constructor con datos `double[][] datosOriginales` aplicando copia defensiva profunda fila por fila.
- Métodos:
  - `public Matriz2D sumar(Matriz2D otra)`: devuelve una nueva matriz resultado (si las dimensiones coinciden).
  - `public Matriz2D transponer()`: devuelve una nueva matriz transpuesta.
  - `public double traza()`: suma de los elementos de la diagonal principal (solo si es cuadrada).
  - `toString()` formateando la matriz con saltos de línea y tabuladores.

Ejercicio 12: Factorías Estáticas (*Static Factory Methods*) con `ColorRGB`

Enunciado:
Crea una clase `ColorRGB`:

- Atributos `private final int rojo`, `verde`, `azul` (todos en el rango [0, 255]).
- **Constructor privado** para impedir la instanciación descontrolada con `new`.
- Constantes predefinidas: `public static final ColorRGB ROJO`, `VERDE`, `AZUL`, `BLANCO`, `NEGRO`.
- Métodos de factoría estáticos:
  - `public static ColorRGB fromRGB(int r, int g, int b)`: valida rangos [0, 255] y devuelve una instancia.
  - `public static ColorRGB fromHex(String hex)`: parsea cadenas como `"#FF5733"` o `"FF5733"`.
  - `public String toHex()`: devuelve la representación hexadecimal en formato `"#RRGGBB"`.

Ejercicio 13: Array de Objetos `Circulo` y Búsqueda del Mayor Área (`mayorArea`)

Enunciado:
Crea la clase `Circulo` con atributo `private double radio`, constructor, getter, `calcularArea()` y `toString()`.

- En `Main`, declara e inicializa un array de 5 objetos `Circulo` con radios distintos (`2.5`, `7.1`, `1.0`, `9.4`, `4.2`).
- Escribe un método estático `public static Circulo mayorArea(Circulo[] circulos)` que recorra el array y devuelva el objeto `Circulo` con mayor área (controlando que el array no sea nulo ni vacío).
- Muestra el círculo ganador y su área por consola.

Ejercicio 14: El Contrato `equals()` / `hashCode()`, la Metáfora de los Cajones y `Punto2D`

Enunciado:

1. **Razonamiento y demostración práctica:** ¿Por qué siempre que se sobrescribe `equals()` es obligatorio sobrescribir también `hashCode()`? ¿Qué ocurre exactamente con los "cajones" (*buckets*) cuando guardas dos objetos iguales según `equals()` en un `HashSet` si olvidaste sobrescribir `hashCode()`? Demuéstralo con código.

2. **Aplicación en `Punto2D`:** Añade a la clase `Punto2D(x, y)` el método `public Punto2D suma(Punto2D otro)`, sobrescribe `toString()` para que devuelva `"(x, y)"`, y sobrescribe `equals(Object obj)` y `hashCode()` anotados con `@Override` para que se pueda imprimir directamente `System.out.println(p1 + " + " + p2 + " = " + p3)` y comparar puntos por valor.

Ejercicio 15: De Clase Mutable `Alumno` a `AlumnoInmutable`

Enunciado:

1. Crea la clase `Alumno` con encapsulamiento: `int nia`, `String nombre` y `double[] notas`. Incluye un constructor con todos los datos, un constructor con solo `nombre` (que genere un `nia` aleatorio y array de 3 notas a `0.0`), getters/setters, `notaMedia()`, `toString()` y `equals()` / `hashCode()` (basados en el `nia`).
2. A continuación, crea la clase `AlumnoInmutable` que sea una versión 100% inmutable de `Alumno` (`final class`, atributos `private final`, sin setters y con copias defensivas `.clone()` del array `notas` en el constructor y en `getNotas()`).

Ejercicio 16: Sistema de Gestión de Inventario: Clases de Dominio, Invariantes y Contenedor de Objetos

Enunciado:
Diseña un sistema orientado a objetos para gestionar el catálogo y stock de un almacén informático mediante clases bien encapsuladas:

1. **Clase de entidad `Articulo`:**
   - Atributos privados: `codigo` (`String`, debe seguir el patrón `^[A-Z]{3}-\d{3}$`, ej. `"TEC-101"`), `descripcion` (`String` no nulo ni en blanco), `precio` (`double > 0`) y `stock` (`int >= 0`).
   - Constructor canónico con validación rigurosa de todas las invariantes (lanzando `IllegalArgumentException` ante datos erróneos).
   - Métodos getters y setters (con validación de datos en setters).
   - Métodos de negocio:
     - `public void reponer(int cantidad)`: incrementa el stock (validando que `cantidad > 0`).
     - `public boolean vender(int cantidad)`: si hay stock suficiente, descuenta las unidades y devuelve `true`; en caso contrario devuelve `false` sin modificar el stock.
   - Sobrescritura canónica de `equals(Object o)` y `hashCode()` basados exclusivamente en el `codigo`.
   - Sobrescritura de `toString()` con formato: `"Articulo[TEC-101: Teclado Mecánico | 59.99 € | Stock: 15]"`.
2. **Clase contenedora de dominio `GestorAlmacen`:**
   - Atributos privados: un array `private Articulo[] articulos` y un contador `private int totalArticulos`.
   - Constructor `public GestorAlmacen(int capacidadMaxima)`: inicializa el array con el tamaño indicado.
   - Métodos del almacén:
     - `public boolean registrarArticulo(Articulo a)`: comprueba que no sea nulo, que el almacén no esté lleno y que no exista ya un artículo con el mismo código (`equals`). Retorna `true` si se registró con éxito.
     - `public Articulo buscarPorCodigo(String codigo)`: busca por código y devuelve la referencia al artículo, o `null` si no existe.

- `public boolean eliminarArticulo(String codigo)`: localiza el artículo, lo elimina y **compacta el array hacia la izquierda** para evitar huecos intermedios, decrementando `totalArticulos`.
- `public double calcularValorTotalInventario()`: calcula y devuelve el valor económico total (`precio * stock`) de todas las existencias almacenadas.
- `public Articulo[] getArticulosDisponibles()`: devuelve una **copia defensiva** del array conteniendo exactamente los artículos registrados (sin posiciones nulas sobrantes).

3. En `main`, crea un almacén con capacidad para 5 productos, registra 3 artículos, realiza una venta válida y otra con stock insuficiente, consulta el valor total del inventario, busca un artículo por código, elimina uno y comprueba que el array queda perfectamente compactado.

> **¿Quieres revisar el código completo?**
> Consulta todas las implementaciones detalladas en el Boletín 4.2 Resuelto.